ROBOTICS CLUB
UNIVERSITY OF SOUTHAMPTON MALAYSIA

RECYCLING CAMPAIGN: AUTOMATED DUSTBIN WORKSHOP PARTICIPANT GUIDE

Part 1: Pre-setup

This section includes information on components and code used in the project to provide the reader with the relevant knowledge to complete the project.

1.1 Component Information

Components:

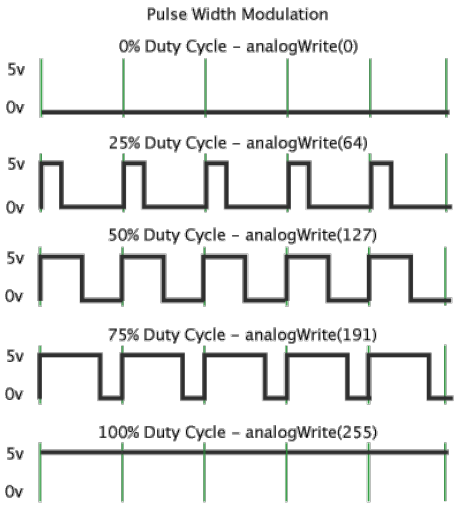
Item	Quantity
sg90 servo 180 degree	1
Ultrasonic sensor	1
Recycled materials	1
10 cm male to female jumper wires	4
10 cm male to male jumper wires	3
Breadboard	1

1) sg90 servo 180 degree

A servo motor is a motor that can rotate and push parts of a machine efficiently and precisely. The output shaft of the motor can be moved to any desired angle or position within its range. It can also be controlled to rotate with a varying velocity unlike a regular DC motor.

The usual servo motor used with an Arduino board is the cost-effective SG-90 9g Micro Servo. It is small, has a range of motion of 180° and is easy to use.

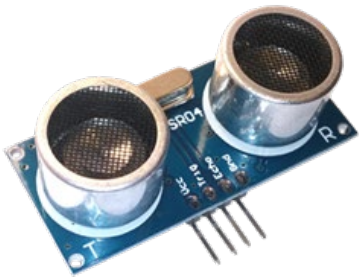
The servo motor is controlled with a Pulse-Width Modulated (PWM) signal as shown below:

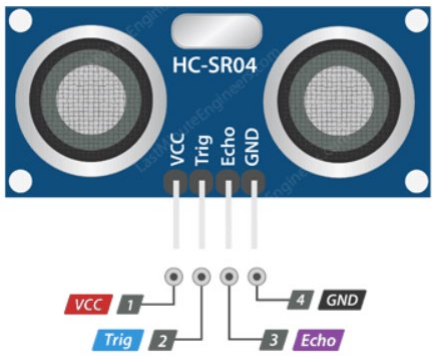


With different PWM values, the shaft rotates to a different position, allowing for easy control using the Arduino board.

The servo motor has 3 different wires: red, brown and yellow which correspond to the power pins 5V, GND and the control PWM pin.

2) Ultrasonic Sensor



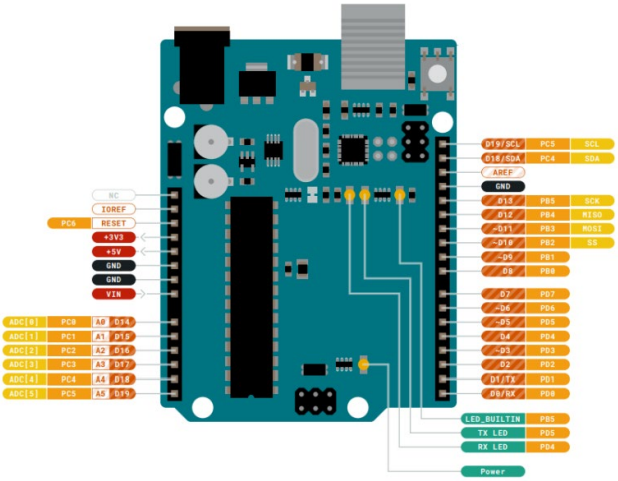


This component detects if an object is nearby. It operates by generating and emitting a sound wave with frequency above human hearing range (ultrasonic wave). It then listens for the reflected pulse signal which indicates the presence of an object in its path. If an object is detected, the sensor returns a signal to the connected Arduino board.

The ultrasonic sensor used in this project is the HCSR04, which has a sensing range of $2 - 400\ cm$.

This ultrasonic sensor has 4 pins: VCC, Trig, Echo, GND. The power pins VCC and GND supply power to the ultrasonic sensor. Trig is used to trigger the ultrasonic sound pulses while Echo produces a pulse when the reflected signal is received.

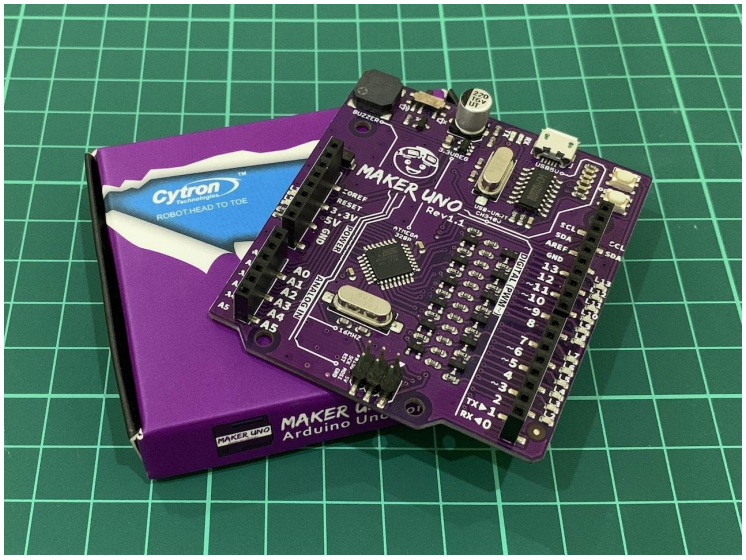
3) Arduino UNO



The Arduino UNO is an easy-to-use programmable board having 20 programmable GPIO (General-Purpose Input/Output) pins, each one having its own functions. Arduino has its own IDE with comprehensive libraries and plenty of example codes that helps beginners to build their own projects. This board uses the ATmega328P microcontroller, the details of which can be found from this link:

https://pdf1.alldatasheet.com/datasheet-pdf/view/313656/ATMEL/ATmega328P.html

4) Maker UNO



Maker UNO is a user-friendly development board like the Arduino UNO, created by Cytron Technologies. It's ideal for beginners to learn electronics and programming. It features built-in LEDs, buttons, and some sensors. It is compatible with the Arduino IDE, and uses the ATmega328 microcontroller too. It has expansion headers, onboard LEDs/buttons, USB

interface, voltage regulator, and is suitable for educational purposes and prototyping on breadboards.

1.2 Project Source Code

1.2.1 Arduino code

In any Arduino project, prewritten code or code written by someone else – "libraries" can be imported to the sketch using the keyword `#include`, saving the trouble of writing code from scratch.

```
// Includes the Servo library
#include <Servo.h>.
```

In this project, the Arduino `<Servo.h>` library is used to assist us in controlling the servo motors.

At the beginning of the code, it is useful to declare global variables or functions that will be used repeatedly. Global variables are variables that can be read and accessed by all functions of a sketch.

```
//Defines Tig and Echo pins of the Ultrasonic Sensors
const int trigPin = A5; //read only, use less memory compare to "int"
const int echoPin = A4;

//Variables for the duration and the distance
long duration; //travelled time
float distance;
float soundSpeed = 0.0343;
```

For the servo motors to work, an object from the "Servo" class has to be instanced. It is named "`myServo`" in this project. The instance can then be used to call functions written in the `<Servo.h>` library.

```
Servo myServo; // Creates a servo object for controlling the servo motor
```

detectDistance() function

```
void detectDistance() {

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH); //return value of time which is high
  distance = duration * soundSpeed / 2;  //in microsec
}
```

The `detectDistance()` function first triggers the ultrasonic sensor to send a pulse of high frequency sound waves by keeping pin 10 high (logic 1) 10 ms. The function "`delayMicroseconds(10)`" is used to introduce delay prior to signal switching. Pin 10 is then pulled to low (logic 0).

Then, the `pulseIn()` function writes pin 11 to HIGH(), commanding the ultrasonic sensor to detect any reflected waves. The time taken for the wave to be detected after its emission is returned and stored in the variable `distance`.

Calculation of the distance is done by multiplying half of the duration by the speed of sound (in $cm/\mu s$). The duration is divided by two because the ultrasonic wave has to travel back and forth. The calculated distance is returned as an integer when the function exits.

setup() function

In the Arduino IDE, the `setup()` function is ran once and only once at the start of the program. It is useful to define initial environment properties or functions that are executed only once within this function (e.g., setting up GPIO pins, initializing serial communication protocol etc).

```
void setup() {
  pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
  pinMode(echoPin, INPUT); // Sets the echoPin as an Input
  Serial.begin(9600);
  myServo.attach(12); // Defines on which pin is the servo motor attached
}
```

`pinMode()` is used to configure specified pins to behave as an input or output.

`Serial.begin(9600)` sets the baud rate (bit rate) of the serial port to 9600, allowing us to establish a serial communication between the Arduino board and the serial monitor in the IDE.

`Servo.attach()` attaches the servo instances to their relevant pins, setting them up to be used for later.

loop() function

In the Arduino IDE, the `loop()` function is repeatedly called after the `setup()` function, this function create what is known as the the main event-loop of the program.

```
void loop() {
  // default servo angle for lid close
  servo1.write(180);

  // detect object distance
  detectDistance();

  // remain the lid open while object distance is less than 10 cm
  while (distance < 10){
    servo1.write(120); // servo angle for lid open
    detectDistance(); // detect the kbject distance again
  }
}
```

The "`Servo.write()`" function move the servo rotation to a specified angle. The function is used here to rotate the servo motor to an initial value at an angle for the lid of the dustbin to close.

The "`detectDistance()`" function from before is called.

"`while (distance <10)`" checks if the distance is smaller than 10 cm. If <10cm, "`Servo.write(120)`" move the servo to 120 degree to open the lid. Then, "`detectDistance()`" to get the latest object distance to see whether to stay in the loop or exit.

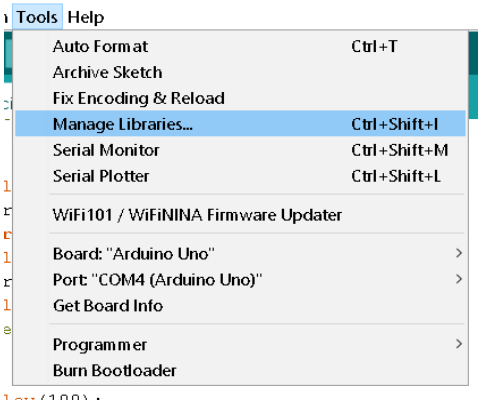
Part 2: Setup

This section provides a step-by-step guide on setting up the Arduino IDE, installing libraries and setting up Processing 4.

2.1 Library installation

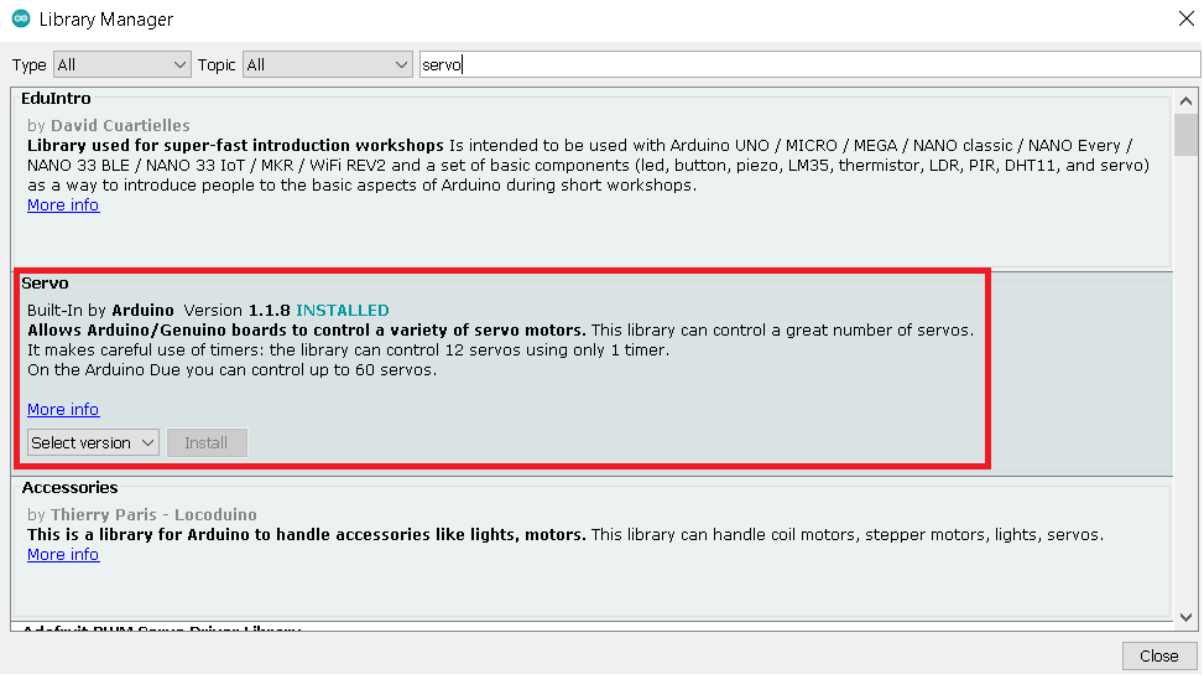
Throughout this project, we will be using 1 external library, namely `<Servo.h>` as stated before.

Go to **Tools → Manage Libraries**.



A window will pop up, allowing us to install and manage our libraries for Arduino.

Type in “Servo” in the search bar and look for the one in the red box, as shown below:



Select the latest version and click install, the library can now be used for all future projects.

Part 3: Assembly

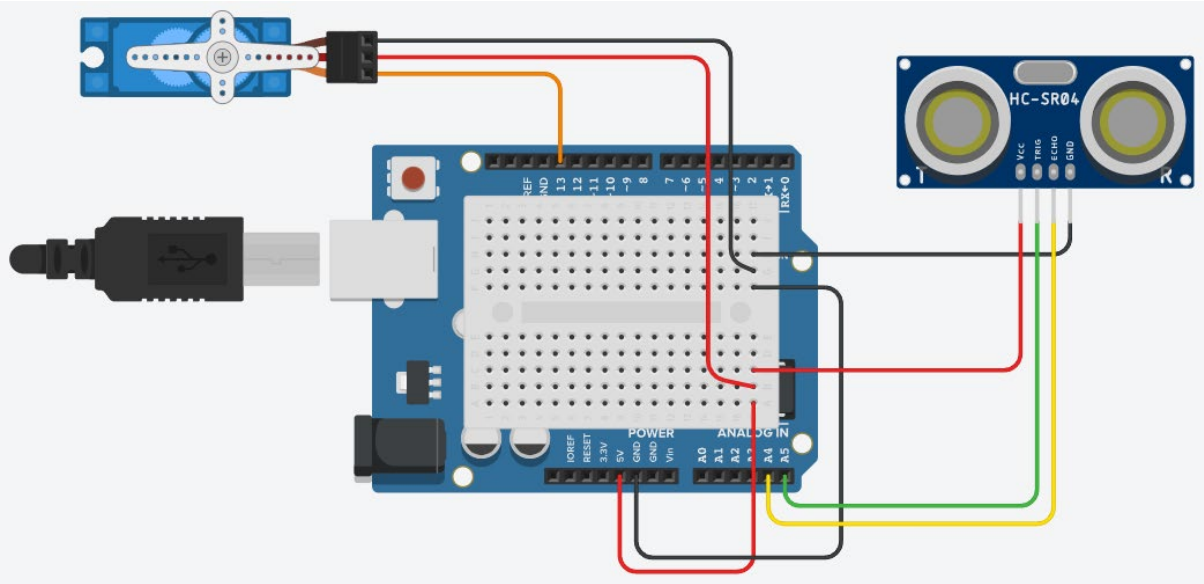
This section will guide you on the full assembly of the project. Follow them closely to get the radar working!

3.1 Parts & Components Checklist

1. sg90 servo 180 degree [x1]
2. Ultrasonic sensor [x1]
3. Maker UNO [x1]
4. Recycle materials

3.2 Circuit diagram

The circuit diagram of the project is as shown:



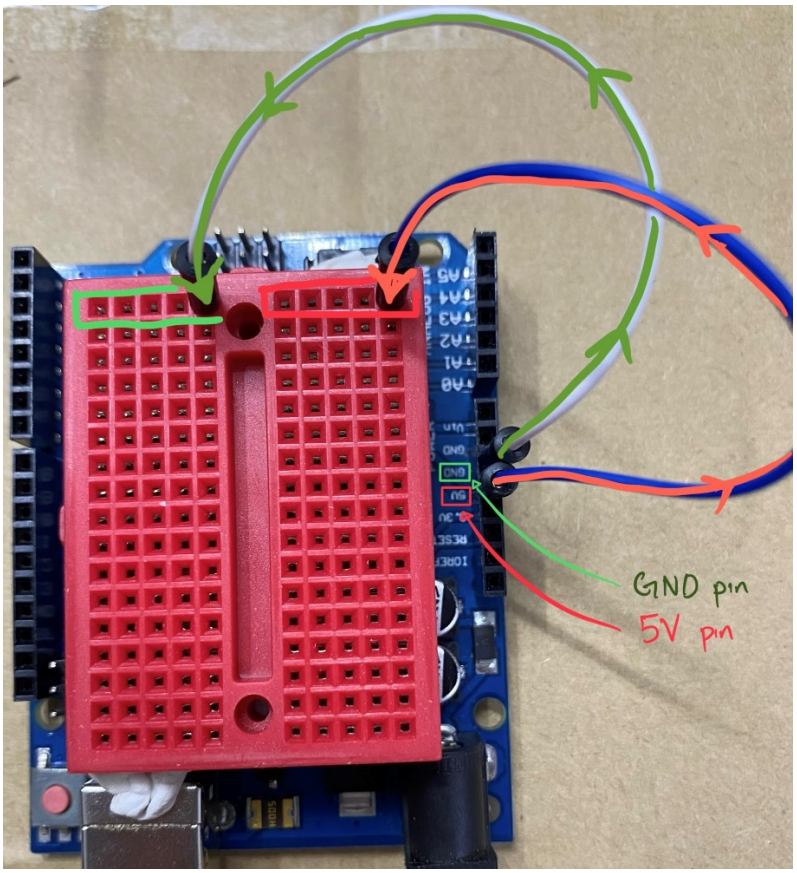
3.3 Assembly

Step 1: Build Your Dustbin

Use the provided tools and materials to build the structure of your dustbin. There is no formula for this, make it unique and pretty!

Step 2: Power Rails

Connect the GND and V5 pins to the bread board as shown below. Then, stick the microcontroller board on to your dustbin using blue tag provided.



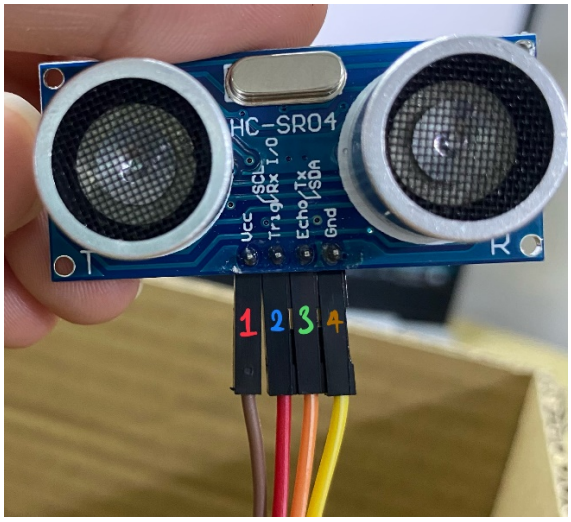
Explanation: As it sounds, the "power rails" are to power the components. All components have to be connected to a power source [5V pin] and to a reference ground [GND pin]. Since we only have one 5V pin on board, we have to "extend" the power in parallel for the ultrasonic sensor and servo motor using a bread board.

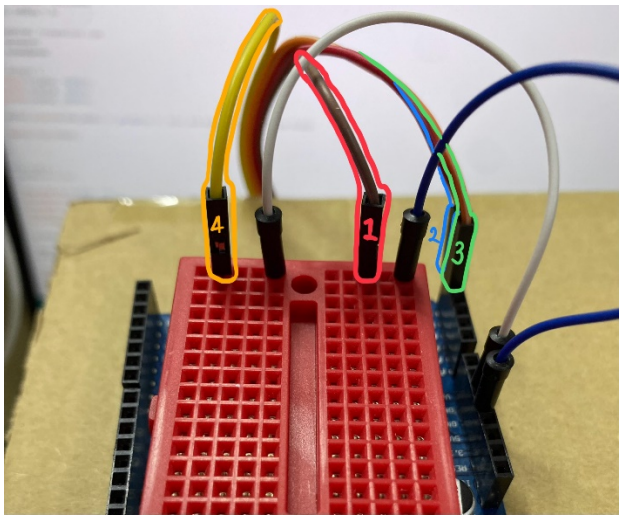
Step 3: Ultrasonic Sensor

This is the fun part >:) – Electronics.

Refer to the schematic at section 3.2 for a full view.

Connect four male-to-female jumper wires to the ultrasonic sensor pins (GND, TRIG, ECHO, VCC) as shown in the picture below. *Tips: it is always a good practice to use different colours wire for different pins for easier hardware debugging.*





Now, Connect the other end of the jumper wires to the microcontroller unit. Follow the following pin assignment carefully.

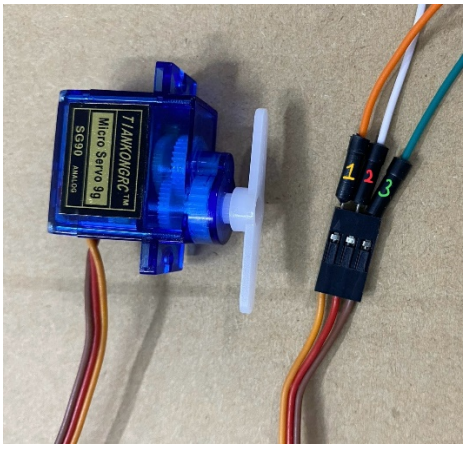
Ultrasonic sensor pin	Microcontroller pin
TRIG	A5
ECHO	A4

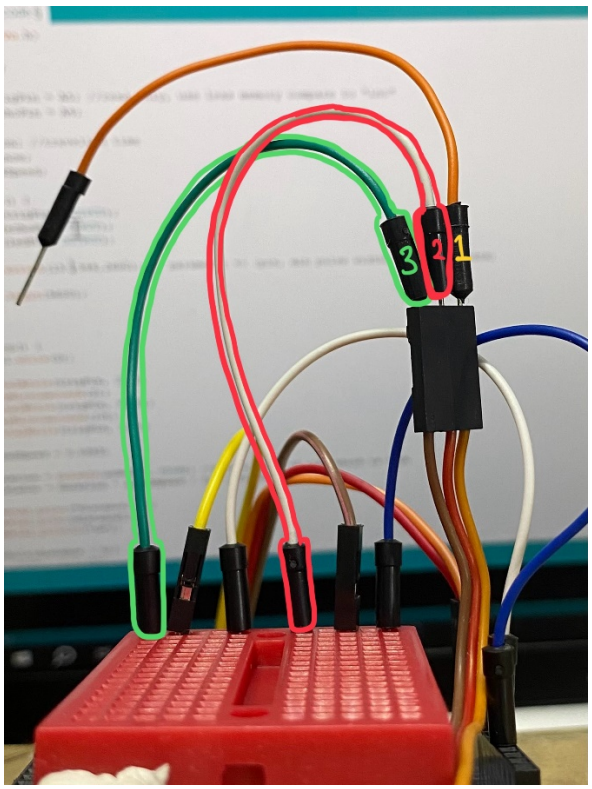
Remember to connect the GND and VCC pins on the breadboard “power rail” as shown in the picture above.

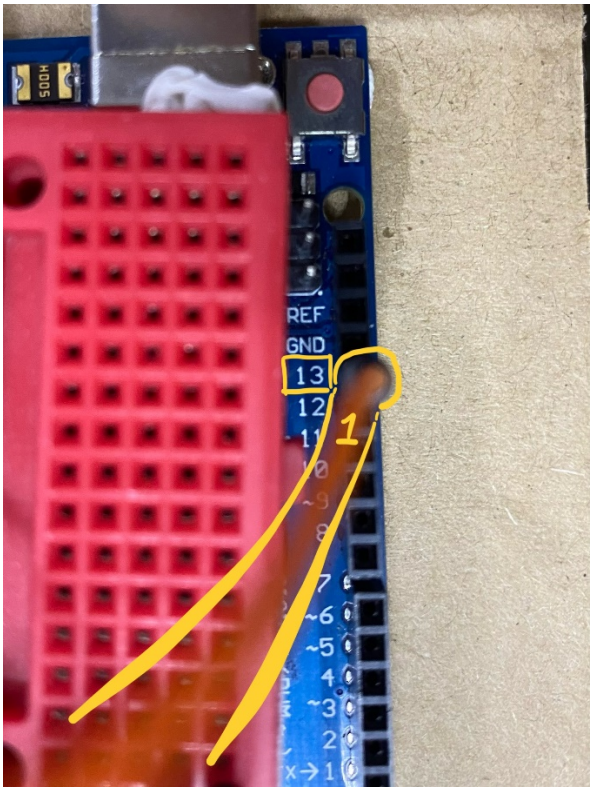
Then, attach the ultrasonic sensor to the structure of your dustbin.

Step 4: Servo Motor

Connect three male-to-male jumper wires to the servo motor (GND, CONTROL, VCC) as shown in the picture below. *Tips: it is always a good practice to use the same colours wire for the same purpose pins. Eg. Red for VCC and Black for GND.*







Now, Connect the other end of the jumper wires to the microcontroller unit. Follow the following pin assignment carefully.

Ultrasonic sensor pin	Microcontroller pin
CONTROL	13

Remember to connect the GND and VCC pins on the breadboard “power rail” as shown.

Step 5: Attach the Servo Motor to the Lid of Dustbin

For this part, it is not necessary to follow the prototype. You are free to decide and build how you want to lift the lid of your dustbin using the servo.

Stick the servo motor close to the top of the dustbin on the back with the provided blue tag.

Attach the string provided to the servo arm and the lid of the dustbin. Make sure the string is not too loose.

Feel free to come forward to the front to have a close up look at the sample prototype we did!

Step 6: Testing

Move your hand or any object close to the ultrasonic sensor and see if the lid of your dustbin lifts up. If it does…

Yay, congratulations you have completed this workshop!!! Show it to any of the committees once you are satisfied with the end product.

:DDD